

9.apriltag标签码识别

9.apriltag标签码识别

9.1简介

9.2 启动

程序启动前的准备

程序说明

程序启动

源码

9.1简介

Apriltag是一种常用于机器视觉中的编码标志，它具有较高的识别率和可靠性，可用于各种任务，包括增强现实、机器人和相机校准。

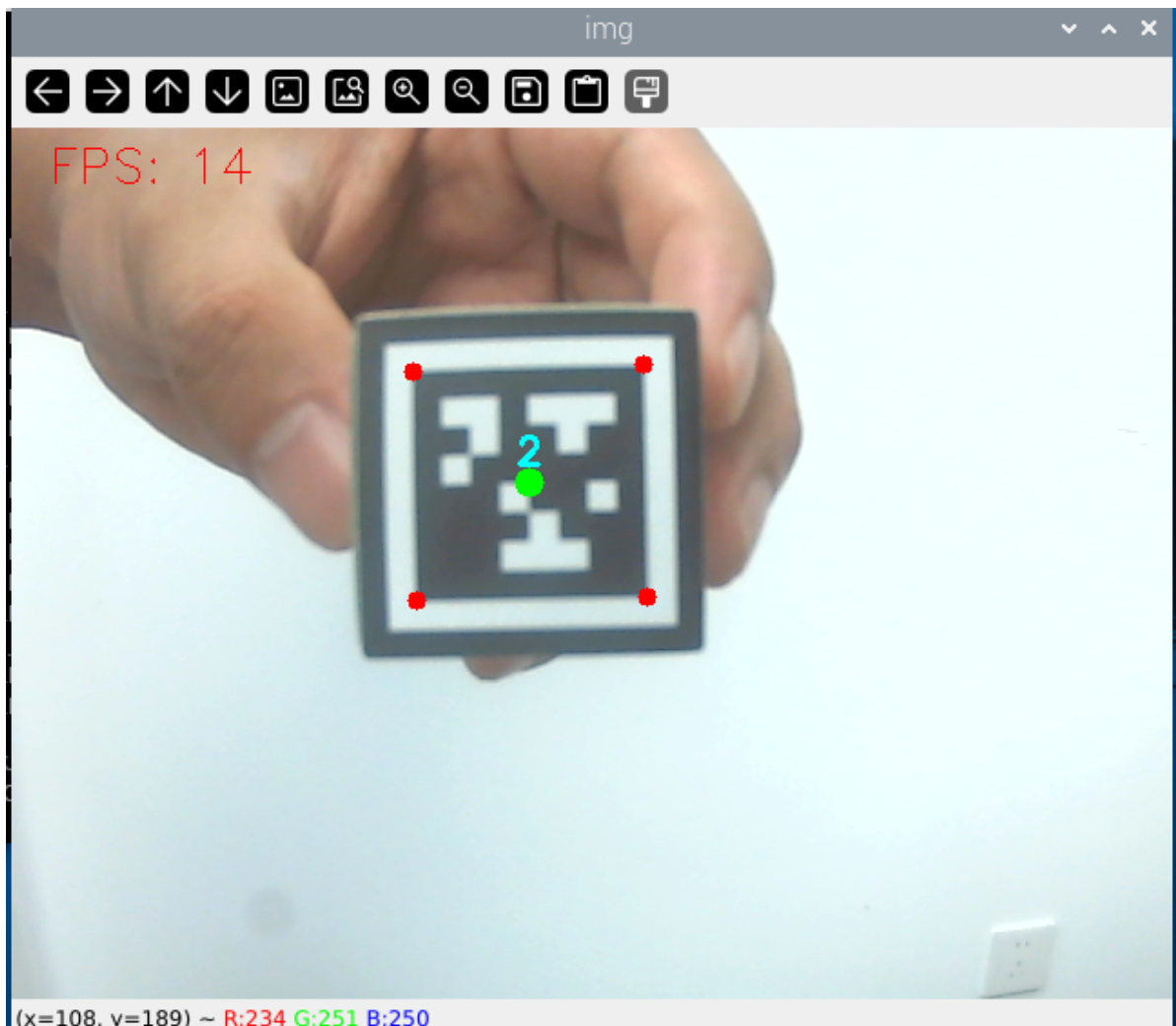
9.2 启动

程序启动前的准备

本次apriltag标签码使用的是TAG36H11格式，出厂已配套相关标签码，并贴在积木块上，需要将积木块拿出来放置到摄像头画面下识别。

程序说明

程序启动后，摄像头捕获到图像，将标签码放入摄像头画面，系统会识别并框出标签码的四个顶点，并显示标签码的ID号。



程序启动

打开一个终端输入以下指令进入docker,

```
./docker_ros2.sh
```

出现以下界面就是进入docker成功

```
pi@yahboom:~ $ ./docker_ros2.sh
access control disabled, clients can connect from any host
root@yahboom:/#
```

在docker终端输入以下命令启动程序

```
ros2 run yahboomcar_apriltag apriltag_identify
```

源码

进入docker容器后, 该功能源码的位置位于,

代码路

径: /root/yahboomcar_ws/src/yahboomcar_apriltag/yahboomcar_apriltag/apriltag_identify.py

```
#!/usr/bin/env python3
# encoding: utf-8
import cv2 as cv
import time
from dt_apriltags import Detector
```

```

from yahboomcar_apriltag.vutils import draw_tags
import logging
import yahboomcar_apriltag.logger_config as logger_config
import rclpy
from rclpy.node import Node
from std_msgs.msg import String, Float32MultiArray

class ApriltagIdentify(Node):
    def __init__(self):
        super().__init__('apriltag_identify_node')
        logger_config.setup_logger()
        self.image = None
        self.at_detector = Detector(searchpath=['apriltags'],
                                    families='tag36h11',
                                    nthreads=8,
                                    quad_decimate=2.0,
                                    quad_sigma=0.0,
                                    refine_edges=1,
                                    decode_sharpening=0.25,
                                    debug=0)

        # AprilTag positions
        self.publisher_ = self.create_publisher(Float32MultiArray,
        'apriltag_positions', 10)
        # AprilTag ID
        self.single_tag_id_publisher_ = self.create_publisher(String,
        'single_apriltag_id', 10)

    def getApriltagPosMsg(self, image):
        self.image = cv.resize(image, (640, 480))
        msg = Float32MultiArray()
        try:
            tags = self.at_detector.detect(cv.cvtColor(
                self.image, cv.COLOR_BGR2GRAY), False, None, 0.025)
            tags = sorted(tags, key=lambda tag: tag.tag_id)
            if len(tags) > 0:
                for tag in tags:
                    point_x = tag.center[0]
                    point_y = tag.center[1]
                    (a, b) = (round(((point_x - 320) / 4000), 5),
                             round(((480 - point_y) / 3000) * 0.7+0.15, 5))
                    msg.data.extend([tag.tag_id, a, b])

                    self.image = draw_tags(self.image, tags, corners_color=(
                        0, 0, 255), center_color=(0, 255, 0))
        except Exception as e:
            logging.info('getApriltagPosMsg e = {}'.format(e))

        return self.image, msg

    def getSingleApriltagID(self, image):
        self.image = cv.resize(image, (640, 480))
        tagId = ""
        try:
            tags = self.at_detector.detect(cv.cvtColor(
                self.image, cv.COLOR_BGR2GRAY), False, None, 0.025)
            tags = sorted(tags, key=lambda tag: tag.tag_id)
            if len(tags) == 1:

```

```

        tagId = str(tags[0].tag_id)
        self.image = draw_tags(self.image, tags, corners_color=(
            0, 0, 255), center_color=(0, 255, 0))
    except Exception as e:
        logging.info('getSingleApriltagID e = {}'.format(e))

    return self.image, tagId

def detect_and_publish(self):
    capture = cv.VideoCapture(0)
    capture.set(cv.CAP_PROP_FRAME_WIDTH, 640)
    capture.set(cv.CAP_PROP_FRAME_HEIGHT, 480)

    t_start = time.time()
    m_fps = 0
    try:
        while capture.isOpened():
            action = cv.waitKey(10) & 0xFF
            if action == ord('q'):
                break
            ret, img = capture.read()
            img, data = self.getApriltagPosMsg(img)

            # AprilTag positions
            self.publisher_.publish(data)

            # AprilTag ID
            _, single_tag_id = self.getSingleApriltagID(img)
            if single_tag_id:
                single_tag_msg = String()
                single_tag_msg.data = single_tag_id
                self.single_tag_id_publisher_.publish(single_tag_msg)

            m_fps += 1
            fps = m_fps / (time.time() - t_start)
            if (time.time() - t_start) >= 2000:
                t_start = time.time()
                m_fps = fps
            text = "FPS: " + str(int(fps))
            cv.putText(img, text, (20, 30), cv.FONT_HERSHEY_SIMPLEX, 0.9,
(0, 0, 255), 1)
            cv.imshow('img', img)
        except Exception as e:
            logging.error('Exception occurred: {}'.format(e))
        finally:
            capture.release()
            cv.destroyAllWindows()

def main(args=None):
    rclpy.init(args=args)
    apriltag_identify = ApriltagIdentify()
    try:
        apriltag_identify.detect_and_publish()
    except KeyboardInterrupt:
        pass

if __name__ == '__main__':
    main()

```

